

# KV Deadline Scheduler: Intent-Aware Memory Management for Long-Context LLM Inference

Manish Klach  
`kv_deadline_scheduler`

June 18, 2026

*Preprint — June 2026. Feedback welcome.  
Target venues: USENIX ATC 2027 / EuroSys 2027 / OSDI 2027.*

## Abstract

Long-context LLM inference creates a new class of memory pressure where generic LRU eviction causes catastrophic decode latency spikes. We present KV Deadline Scheduler, a system that propagates request-level semantic intent—such as deadline, phase, priority, and recompute cost—from the serving layer to the memory manager. In our pressure-calibrated policy simulator, intent-aware policies reduce decode-critical eviction rates from 4.494% under LRU to 0.281% under IntentAware and DeadlineAware scheduling at 12× effective HBM pressure.<sup>1</sup> We complement the simulator with Linux-focused experiments covering `madvise`, THP, `perf_event_open`, `raw io_uring`, and userspace I/O priority separation, then outline a kernel integration path that spans shrinker prototypes, DAMON-facing instrumentation, and an RFC memory-intent registry. We also propose the Memory Intent ABI, a versioned runtime-to-OS contract for exposing structured memory semantics in both JSONL traces and low-latency shared-memory paths. Finally, we extend the design toward disaggregated KV cache and speculative decoding, two settings where semantic scheduling becomes even more important. The result is a research platform for bridging LLM runtimes and operating-system memory management around a single thesis: KV cache is not anonymous memory.

## 1 Introduction

Long-context inference makes memory movement as important as arithmetic throughput. A 128k-token request can create large, persistent KV state, and a serving stack with tens or hundreds of concurrent requests quickly

<sup>1</sup>Under a higher-pressure synthetic run (16 requests, 12 blocks, 512 KB/block), LRU reached 12.4% DC miss rate, confirming the trend.

turns fast memory into a contested resource. Generic page-based eviction is blind to semantic differences between a cold prefill block and a decode-critical block that must be present before the next token deadline. That blindness causes tail-latency spikes and can trigger avoidable recompute.

KV Deadline Scheduler starts from a simple observation: the serving layer already knows far more than the memory manager. It knows which block belongs to which request, what phase that request is in, how urgent the next decode step is, whether the block is speculative, and how expensive recompute would be. Our work packages that knowledge into a structured memory-intent interface and uses it to drive both simulation and Linux-facing experiments.

Our contributions are:

1. A formal Memory Intent ABI for runtime-to-OS memory semantics.
2. A policy ladder from LRU to DeadlineAware scheduling with real pressure-calibrated results.
3. A Linux experiment suite spanning VM, I/O, and kernel-facing prototypes.
4. A generic memory-intent RFC track for Linux MM, designed around observability first.
5. Extensions for disaggregated KV scheduling and speculative decoding.

## 2 Background

Transformer inference stores keys and values for past tokens so future attention can reference them without re-computation. Systems such as vLLM, which introduced PagedAttention [1, 2], make this practical by chunking and reusing KV storage, but they still rely on lower layers that largely see memory as anonymous pages or buffers.

Linux MM offers several relevant mechanisms: classic LRU-like reclaim, MGLRU [7], shrinkers, `madvise`, and

Table 1: Selected Memory Intent ABI fields used by the current prototype.<sup>2</sup>

| Field                          | Meaning                   | Units   |
|--------------------------------|---------------------------|---------|
| <code>deadline_us</code>       | decode or fetch deadline  | $\mu s$ |
| <code>slack_us</code>          | time until deadline       | $\mu s$ |
| <code>recompute_cost_us</code> | local rebuild penalty     | $\mu s$ |
| <code>spill_cost_us</code>     | offload or recall penalty | $\mu s$ |
| <code>last_access_step</code>  | recency proxy             | steps   |

DAMON-based observation [8]. Prior KV offload systems such as LMCache [3], Mooncake [4], MemServe [5], and Infinite-LLM [6] focus on *where* KV lives and how it is fetched. Our focus differs: we ask how semantic intent should flow from the inference runtime into the memory manager.

### 3 Design

The central abstraction is `memory_intent_t`, a structured description of a memory object. It includes identity, phase, priority, deadlines, slack, allowed tiers, recompute cost, spill cost, recency, and flags such as `PIN_REQUESTED`, `RECOMPUTE_OK`, and `IS_DRAFT`.

The policy ladder is intentionally incremental:

**LRU** uses recency only.

**HotCold** adds inferred hotness.

**PredictiveHotness** adds a reuse model.

**IntentAware** adds semantic request information.

**DeadlineAware** adds deadline and slack protection explicitly.

We model the intended eviction score as a composition of recency, urgency, and semantic importance:

$$\text{score}(b) = \alpha R(b) - \beta D(b) - \gamma P(b) + \delta C(b), \quad (1)$$

where  $R(b)$  captures recency,  $D(b)$  captures deadline pressure,  $P(b)$  captures semantic priority such as decode-critical state, and  $C(b)$  captures reclaim or recompute cost. Lower scores are harder to evict. In the current implementation,  $D(b)$  and  $P(b)$  are computed using threshold scoring rather than continuous weights—for example, a deadline under 1 ms contributes a fixed  $-320$  to the eviction score, while `DECODE_CRITICAL` priority contributes  $-400$ . This threshold design is deliberately simple and auditable; the continuous formula above describes the design intent.

<sup>2</sup>The ABI uses microsecond resolution as sufficient for KV scheduling granularity; nanosecond precision can be added in a future ABI version without breaking existing consumers.

Table 2: Decode-critical miss rates across policies under high pressure.

| Policy            | DC Miss Rate | Reduction vs. LRU |
|-------------------|--------------|-------------------|
| LRU (baseline)    | 4.494%       | —                 |
| HotCold           | 4.494%       | 0%                |
| PredictiveHotness | 4.494%       | 0%                |
| IntentAware       | 0.281%       | 94%               |
| DeadlineAware     | 0.281%       | 94%               |

The Memory Intent ABI has two wire forms: JSONL for traces and a shared-memory ring for low-latency paths. The kernel integration path mirrors this layering: start with registry and observability, then add DAMON reporting, and only later consider default-off reclaim behavior.

### 4 Implementation

The Python simulator is event-driven and tracks per-block tier placement, latency penalties, spills, prefetches, misses, and decode-critical evictions. The Linux-facing artifacts include a shrinker prototype, a DAMON classification sketch, debugfs-based memory-intent RFC patches, and a set of user-space experiments covering VM and I/O behavior.

### 5 Policy Evaluation

The core result comes from the pressure-calibrated sweep under the committed `deadline_pressure` profile with seed 42. At  $12\times$  effective HBM pressure, recency-only and hotness-only policies fail to protect decode-critical request-state. LRU, HotCold, and PredictiveHotness each reach a 4.494% decode-critical miss rate, while IntentAware and DeadlineAware reduce that rate to 0.281%. The main lesson is not merely that deadline-aware scheduling helps, but that semantic request knowledge matters most when pressure is high enough that recency stops being a reliable proxy for urgency.

Under this workload shape, the semantics-free policies converge because they all observe similarly overloaded access streams. HotCold and PredictiveHotness show no reduction at  $12\times$  pressure because they lack semantic phase and deadline knowledge. The threshold where they help is lower-pressure scenarios where recency remains a reasonable proxy for reuse. That is an honest finding rather than a weakness to hide: semantic policies earn their advantage specifically in the regime where page-trace heuristics become ambiguous.

## 6 Linux Experiment Layer

The Linux experiment layer provides supporting evidence rather than end-to-end serving proof. On WSL, I/O priority separation reduced worst-case critical-read latency. THP showed a positive throughput signal. `perf_event_open` showed much higher cache-miss rates for KV-random and evicted patterns than for sequential access. Together, these experiments reinforce the broader claim that long-context inference is constrained not only by compute, but by a stack of VM, cache, and storage behaviors that generic infrastructure was not designed to interpret semantically.

The raw `io_uring` path operated correctly end-to-end on the WSL storage stack (p50 = 332  $\mu$ s, p95 = 1,495  $\mu$ s, p99 = 2,378  $\mu$ s for 256 KB KV blocks). Throughput improvement over synchronous reads was not observed under WSL virtualised storage—an expected result, since Hyper-V does not enforce `io_uring` queue depth semantics between processes. Bare-metal NVMe evaluation remains future work.

## 7 Disaggregated KV Extension

Disaggregated KV systems separate prefill, decode, and storage roles across machines. In that setting, the deadline question expands: a block may be semantically important but still unreachable before its deadline if network RTT dominates slack. Our `RemoteAwarePolicy` extends `DeadlineAware` scheduling with network cost, migration-in-flight protection, and a recompute-versus-fetch decision boundary. Rather than asking only whether a block is important, the policy asks whether it can arrive in time to matter.

Consider a three-node cluster consisting of a prefill node, a decode node, and a storage node, modeled after the Mooncake architecture [4]. A KV block committed on the prefill node may later be needed by the decode node with `deadline_us` = 5000. If the measured network RTT to the decode node is 200  $\mu$ s, `RemoteAwarePolicy` approves a remote fetch because the transfer comfortably fits within the slack budget. If the same request experiences congestion and RTT rises to 4,000  $\mu$ s, while the local `recompute_cost_us` is 1,500, the policy instead selects local recompute. This boundary—roughly `deadline_us` < `rtt_us`  $\times$  1.5 in the current simulator—is exposed as a configurable threshold.

This extension matters because semantic urgency alone is insufficient once memory is physically separated from compute. A decode-critical block stored remotely may still be the wrong retrieval choice if transit time erases its usefulness. Conversely, a non-critical block may be worth fetching if it is cheap to move and likely to be reused across several near-future decode steps. The disaggregated model therefore treats network transfer

as part of the effective memory hierarchy and folds it into the same control-plane vocabulary: deadline, slack, priority, and recompute cost.

The broader implication is that semantic memory scheduling can unify local and remote decisions. The same intent fields that distinguish a cold prefill block from verified decode state also help decide whether a remote recall is viable. This is one reason we prefer a workload-neutral memory-intent ABI over a narrow KV-only hint path: once the control plane exists, the scheduler can reason across HBM, DRAM, CXL, NVMe, and network-attached KV pools using the same language of urgency and cost.

## 8 Speculative Decoding Extension

Speculative decoding introduces a different failure mode: memory pressure can be created by state that may never be committed. Our `SpeculativeIntentPolicy` treats draft KV differently from verified decode state by marking it as `IS_DRAFT` and explicitly preventing it from inheriting `DECODE_CRITICAL` protection until verification succeeds. This is semantically important. A speculative branch may be recently accessed and therefore appear “hot” to a generic policy, yet still be the wrong object to preserve if it has not survived token-tree verification.

Consider an EAGLE-2 style speculative decoder [10] with a draft tree of depth 3 and width 4. That configuration can create up to 12 concurrent draft KV blocks in flight. If 8 of those are rejected at the verify step, `SpeculativeIntentPolicy` immediately frees those 8 blocks and recovers their HBM allocation before the next decode step. Under a 50% acceptance rate, this prevents roughly 6 MB of wasted HBM per speculative step, which becomes significant at high concurrency. The committed speculative sweep in this repository shows the same qualitative outcome: verified decode protection is preserved while rejected drafts are reclaimed promptly.

The design principle is straightforward: speculative state should compete with verified state under a different protection budget. Draft blocks can be useful, but they are not equally entitled to scarce HBM. The policy therefore rewards acceptance and verification, not just access. Once a draft survives verification, it becomes eligible for the same deadline-aware protections as ordinary decode-critical state. Until then, it remains cheaper to evict, spill, or discard.

This extension also illustrates why a semantic control plane is preferable to page-trace inference alone. A generic page-based policy sees memory pressure from many recently touched draft buffers and may preserve them too aggressively. A semantic scheduler knows that speculative state is provisional. In effect, the policy prevents unverified branches from crowding out the

request-state that is already on the critical path to the next token.

## 9 Limitations and Future Work

Current results are still dominated by simulation rather than a full GPU runtime. The kernel patches are RFC material, not upstreamed or broadly validated kernel changes. QEMU validation is scaffolded but not yet used as the basis for a published kernel result. The speculative decode extension is policy-complete but not yet calibrated against a production speculative stack.

## 10 Conclusion

KV Deadline Scheduler argues that future AI infrastructure must schedule memory *meaning*, not just memory residency. By formalizing intent, connecting it to simulation and Linux experiments, and exposing a path toward runtime-to-OS integration, the project provides a concrete platform for research at the intersection of LLM inference and operating-system memory management.

## References

- [1] W. Kwon, Z. Li, M. Rabe, et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention.” SOSP 2023. <https://arxiv.org/abs/2309.06180>
- [2] vLLM project. <https://github.com/vllm-project/vllm>
- [3] Y. Zhao, et al. “LMCache: A Knowledge Sharing Service for Efficient LLM Serving.” arXiv:2401.02669, 2024. <https://arxiv.org/abs/2401.02669>
- [4] R. Qian, et al. “Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving.” arXiv:2407.00079, 2024. <https://arxiv.org/abs/2407.00079>
- [5] S. Hu, et al. “MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool.” arXiv:2406.17565, 2024. <https://arxiv.org/abs/2406.17565>
- [6] C. Lin, et al. “Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Disaggregated Memory.” arXiv:2401.02669, 2024.
- [7] Y. Zhao. “Multi-Gen LRU.” Linux kernel 6.1. <https://lwn.net/Articles/851184/>
- [8] S. Park. “DAMON: Data Access MONitor.” Linux kernel 5.15. <https://lwn.net/Articles/812707/>
- [9] T. Cai, et al. “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads.” arXiv:2401.10774, 2024. <https://arxiv.org/abs/2401.10774>
- [10] Y. Li, et al. “EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty.” ICML 2024. <https://arxiv.org/abs/2401.15077>
- [11] X. Miao, et al. “SpecInfer: Accelerating Generative LLM Serving with Speculative Inference and Token Tree Verification.” ASPLOS 2024. <https://arxiv.org/abs/2305.09781>
- [12] CXL Consortium. “Compute Express Link Specification 3.0.” 2023. <https://www.computeexpresslink.org>